

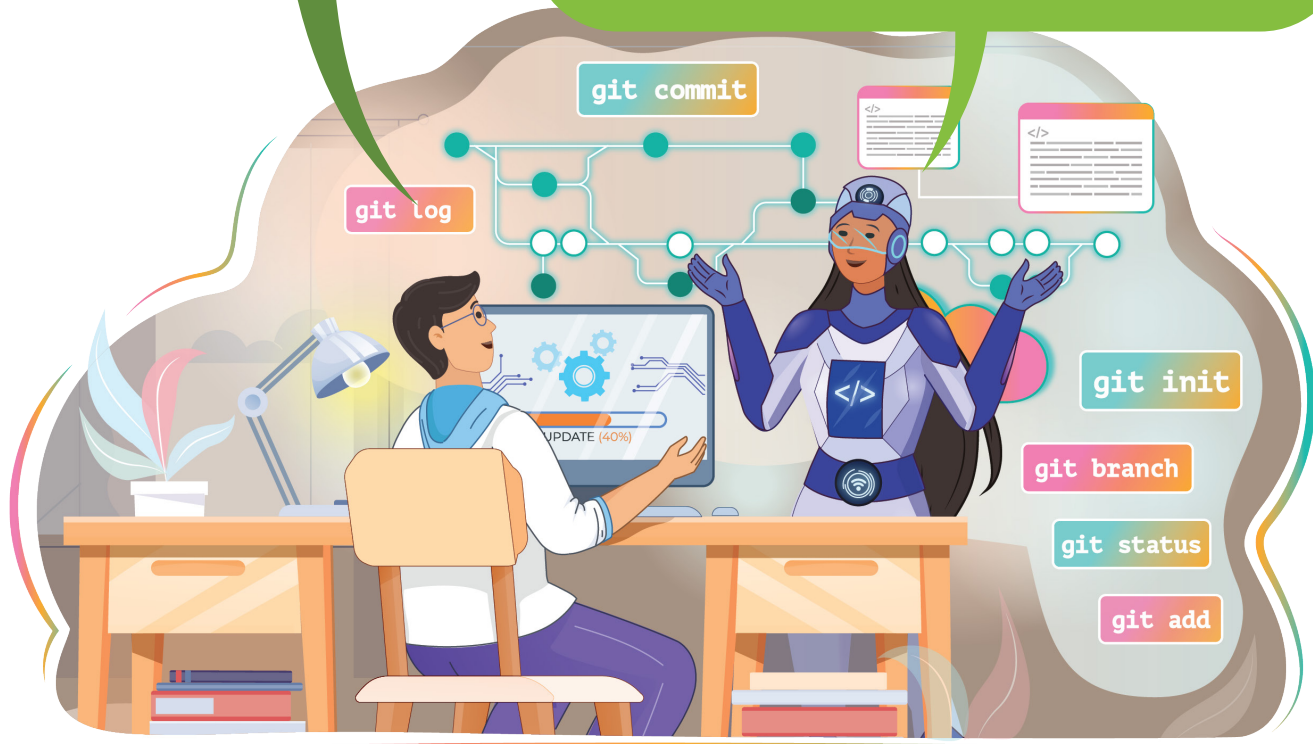
Introduction to Git

03



I know software updates are important and they're done by releasing multiple versions from time to time, but how are these versions managed?

Basically, these are done using special applications known as 'Version Control System'. They store every version on the server, which can be accessed and modified in a collaborative manner as and when needed.



In this lesson, we will get familiar with 'Git' and its application as a 'Version Control System'. Before we understand Git, we must first understand a few concepts, such as 'Software Versions' and 'Version Control Tools'.

'Git' is a significant version control system for software development since it helps developers to keep track of changes made to code, collaborate more efficiently, and revert to previous versions if necessary. Reasons for learning 'Git' include working on a personal coding project or seeking a profession in software development.

What are Application/Software Versions?

Let us assume that we are creating an application/software that will store the information of all the members of the organization as well as perform other tasks, such as adding new members, evaluating their purpose, and so on. However, after a few months, you decide to upgrade your application and add some new features such as maintaining the information of all the products, adding some new application security features, etc. In such cases, when we enhance an existing application, we create a new version of it (let's say Version 2). This new version will contain the new functionalities along with the existing ones.

In this manner, we can keep on adding new functionalities and create different versions (Version 3, Version 4, Version 5, and so on) of the same product. These are referred to as 'Software or Application Versions'.

Technically, **Software Versions** refer to different releases of a software application or program. Developers often release updates to their software to fix bugs, add new features, improve performance, and enhance security. Each release is assigned a version number or code that identifies it uniquely.

For example, applications like your browser and anti-virus software frequently ask about updating to the latest versions to be equipped with all the latest upgrades and changes. Users should normally maintain their software up to date with the most recent version to ensure they are using the safest and most stable version available.

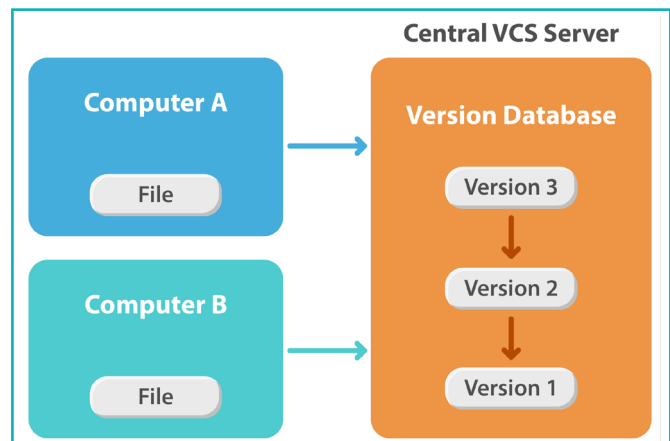
What is Version Control?

While creating versions, it is important to keep track of what changes you made in a particular version. In simple terms, we need to keep track of the versions so that we can revert to a previous version in case there are some issues with the newer versions. Specific applications are available for this purpose, and these are known as **Version Control Applications**. They keep a log of all the changes made to an application, allowing us to evaluate or even restore the application to its previous state.

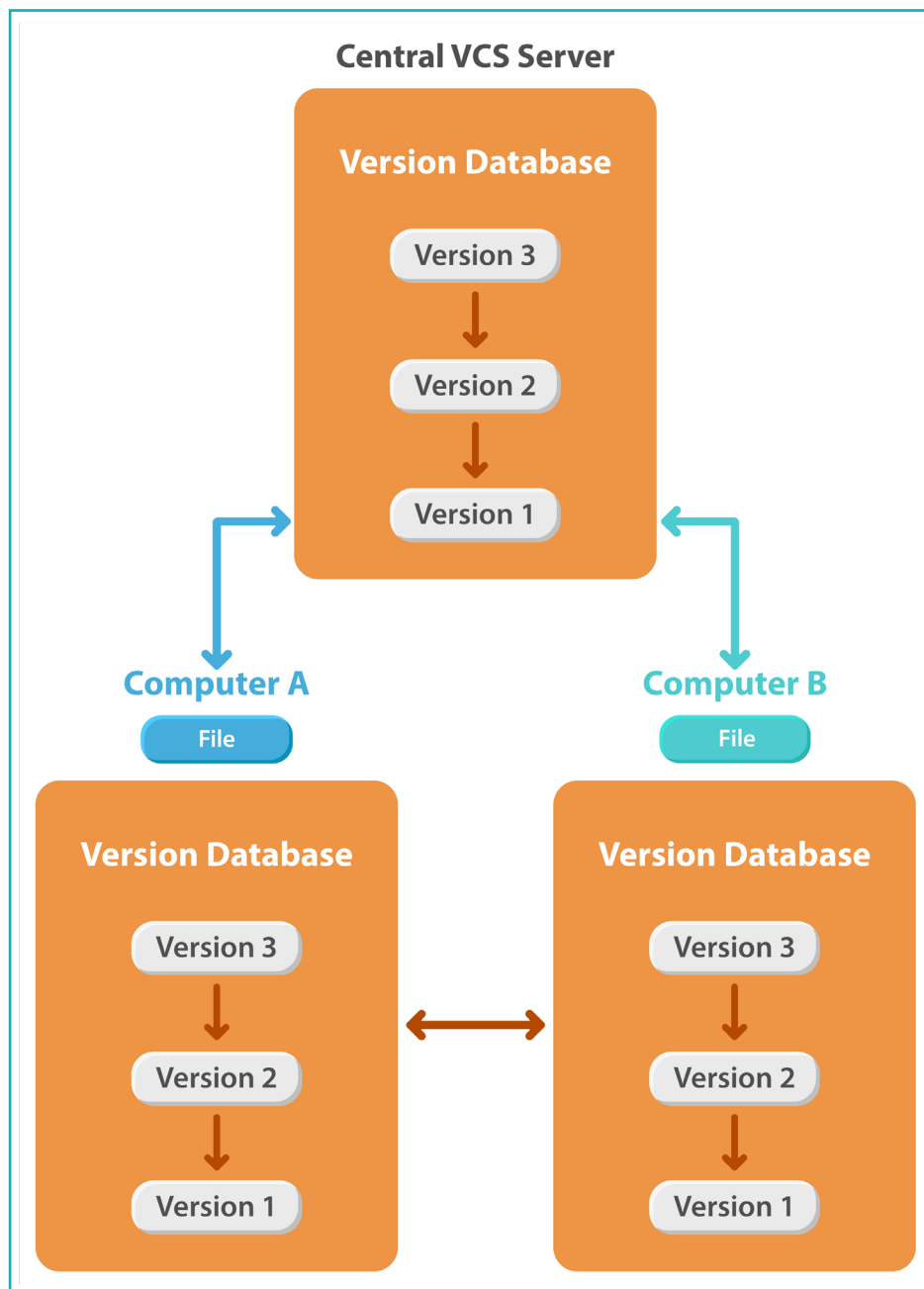
Version Control Systems are broadly classified into 2 categories:

1. **Centralized Version Control Systems (CVCS)**
2. **Distributed Version Control Systems (DVCS)**

In 'Centralized Version Control Systems', all application files and versions are stored on a single server, and multiple users (developers) can access and modify files from that server. When a user wants to modify the application, they simply download the files from the server to their local PC. Following the changes, the files are re-uploaded to the central server. By connecting to the central server, 'CVCS' also allows people working from different locations to collaborate with one another.



The mechanism of the 'Distributed Version Control System' is nearly identical to that of the 'Central Version Control System.' There is also a central server here, but instead of the central server having all the files, each user has their own copy (clone) of the complete repository in their local system, containing all the application files and versions. In simple terms, 'DVCS' has a remote repository that is stored on a server as well as a local repository that is stored on each developer's workplace computers.



Key Differences between CVCS and DVCS

- a. 'DVCS' allows us to work even if we are not connected to the internet, as the entire repository resides on your local system. This is not the case with 'CVCS'.
- b. 'DVCS' is faster than 'CVCS', as we do not have to connect with a remote server. You handle everything on a local level.
- c. Working with 'DVCS' is a bit more complicated than 'CVCS'.
- d. In 'CVCS', the entire system comes to a halt if the central server crashes. This is not the case with 'DVCS', as each user has access to the whole repository, available in their local machine.

There are various 'Version Control' applications such as 'Beanstalk', 'SVN', 'Git', and 'CVS', available. Among them, 'Git' is the most popular one.

Exercise

 Fill in the blanks.

- 1 CVCS stands for _____.
- 2 Git is a _____ Version Control System.

What is Git?

Git is a free and widely used 'Distributed Version Control System' that is used to keep track of changes made to files in software development projects.

A few features of 'Git' are mentioned below:

- a. It is Open Source, which implies that anybody can access, alter, and distribute Git's source code.
- b. Since 'Git' is a 'Distributed Version Control System', it has a remote repository stored on a server and a local repository, stored on the local machine of developers who are working on that repository.
- c. It provides a history of all the changes made to the files, including who made them and when.
- d. It allows for efficient collaboration by allowing multiple team members to work on the same project simultaneously.
- e. 'Git' provides a 'Code Review' process that allows all changes made in the repository by developers to be reviewed by others before they are merged into the central repository.
- f. It also keeps data backups for simple recovery during data loss or system crash.

Why use Git?

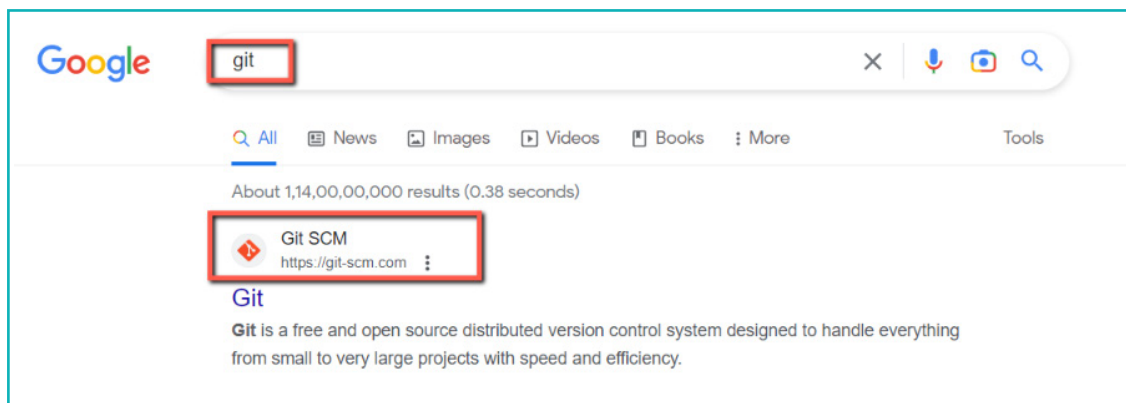
Creating software or a web application is not a one-man job in the real world. Multiple developers are involved, who work in parallel with each other. A Version Control System, such as 'Git', keeps all associated developers up to date on the newest updates to the project. It is also not necessary for all developers to be present at the same time. They can be in different locations and still be aware of the most recent development process because the full code repository is available on their local machine.

In the absence of 'Version Control Systems' like 'Git', such types of collaborative work would be very difficult to implement, as one developer may not be aware of what others are working on. This can be compared to the concept of group projects, where multiple students work together on the same project.

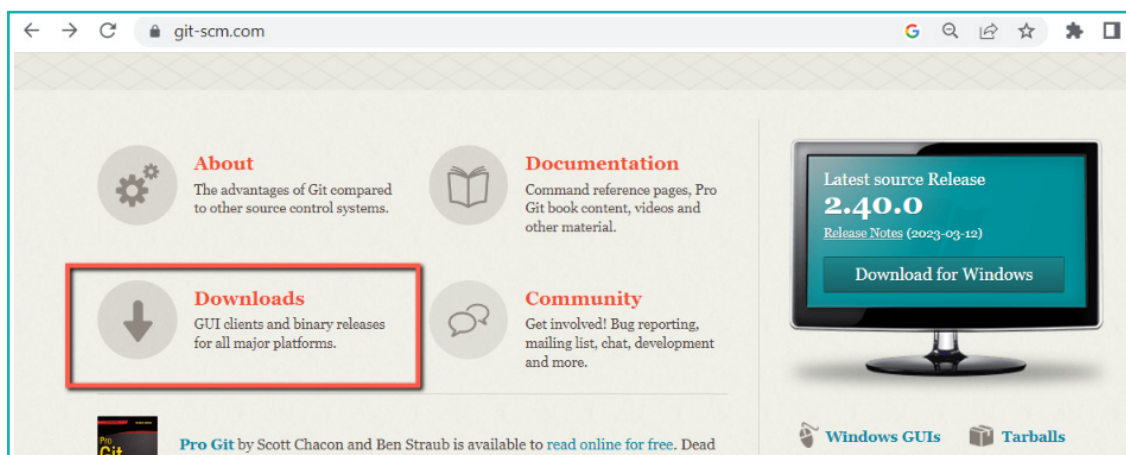
Also, since the entire application development history is maintained, one can always revert to a previous working version of the application, in case of any errors or changes in the project requirements.

Step 1: Installing Git

- 1 Search for 'Git' on Google and open their official website.



- 2 On their official website, click on 'Downloads'.



- 3 Depending on your platform, click on the appropriate 'Downloads' link. For our lesson, we will go ahead with 'Windows'.

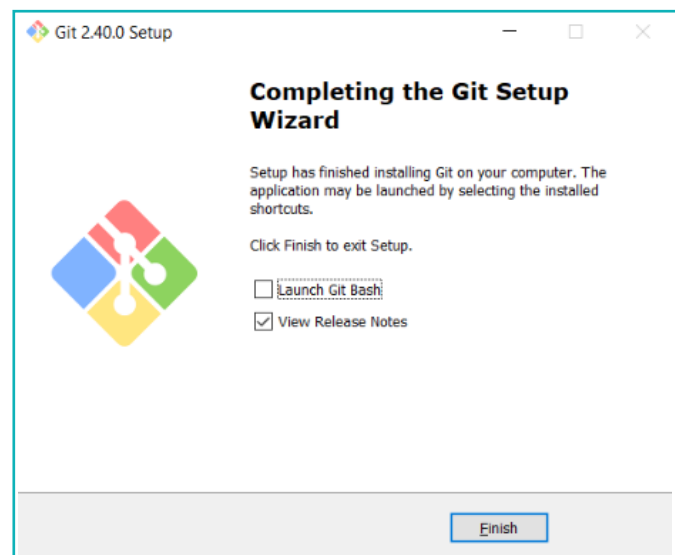


- 4 On the next page, select the '32 bit' or '64-bit' version under the 'Standalone Installer' and download the application.



- 5 Once downloaded, install the application. Keep all the settings to default during the installation process and continue with the installation by clicking on the 'Next' button. Once done, click on the 'Finish' button.

Git is now available on your system.



Exercise

 Tick mark ☒ the correct answer

3 Which of the following statements is not true about Git?

- a. It is Open Source ☐
- b. It is a Decentralized Version Control System ☐
- c. It is a Centralized Version Control System ☐
- d. Git allows us to maintain a history of all changes made within a repository. ☐

4 Which of the following is not true about DVCS?

- a. It is faster than CVCS ☐
- b. Can work even if not connected to the internet. ☐
- c. The entire server comes to halt if the central server crashes. ☐
- d. Git is a type of DVCS ☐

 State if the statements are True or False

5 Git provides a facility for data backup.

 _____

6 Git is available only for the Windows Platform.

 _____

Working with Git Repository

What is a Repository?

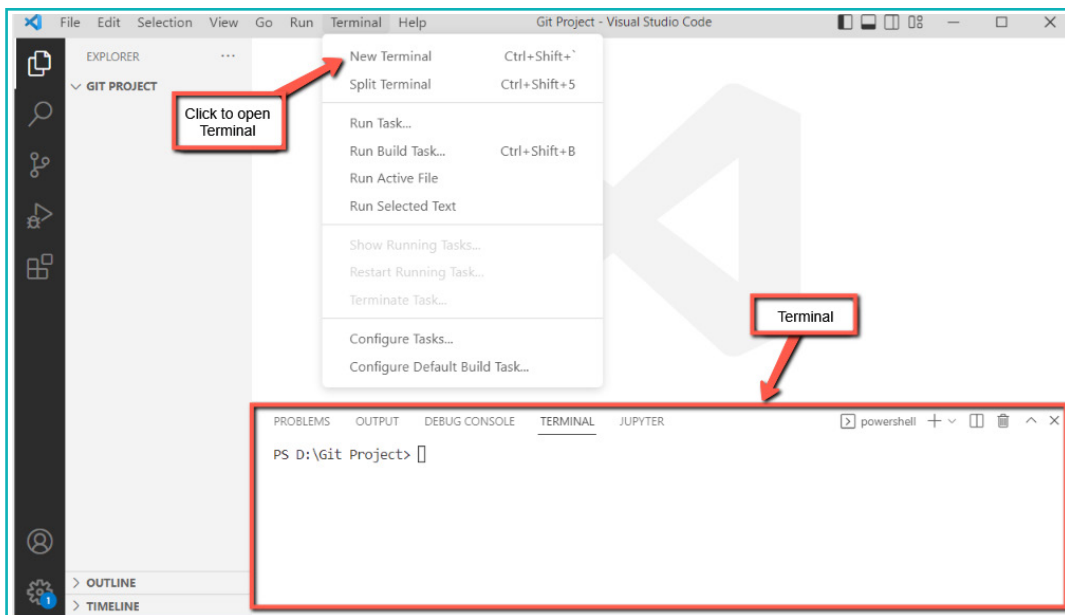
In simple language, a 'repository' is a location, such as a box, or a room, where things are stored for safekeeping. A building where diamonds are held, for example, is a repository for diamonds.

In terms of computer programming, a 'Repository' or a 'Repo' is a centralized location where data, files or documents are organised and saved. Developers use repositories to collaborate on code, share their work with others, and track changes to the code over time.

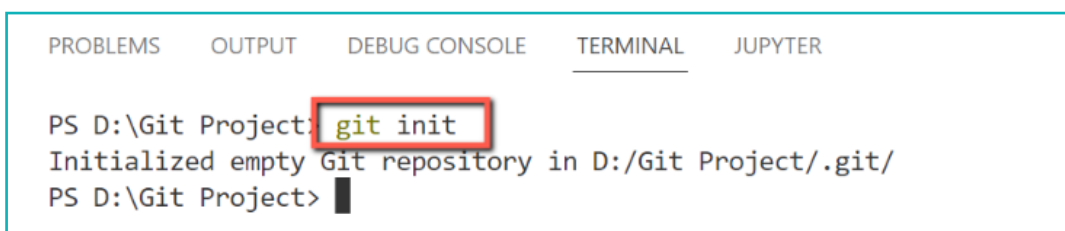
To start using Git, we must first build a Git repository. The simplest way to create one is to take a local application directory or folder and convert it into a Git Repository. It is also referred to as a 'Local Git Repository' or a 'Git Repo', in short.

So, let us create a folder on our local machine and turn it into a Git Repository.

- 1 Create a folder "Git Project" and open it in VS Code. Within VS Code, open the 'Terminal' by clicking on the 'Terminal' menu and selecting 'New Terminal'.

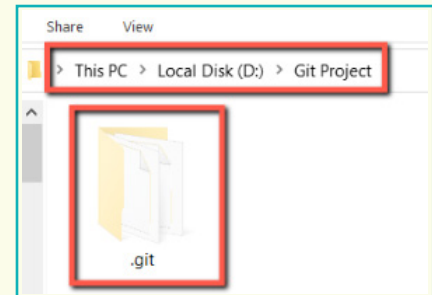


- 2 To turn the local directory into a Git Repository, all we need to do is execute the command **git init**.



Note:

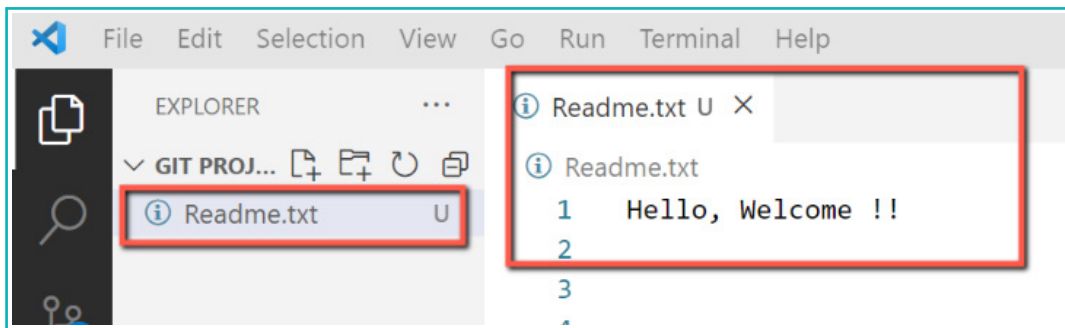
When we initialize a local folder as a Git Repo, a hidden folder with the name `.git` is created, which contains all the git metadata for tracking changes made within the folder.



Now, whatever files or folders we create, along with all the changes we make on those files, will be tracked by 'Git'.

To demonstrate it, we will create a simple text file in the 'Git Project' folder.

- 3 Create a text file 'Readme.txt' and add some text to it.

**Note:**

'Readme' is the file name and '.txt' is the extension.

We currently have a 'Git Repository' with a single text file. However, when we add, modify, or remove files from a 'Git Repo', Git will not track it on its own.

Tracking Files using Git

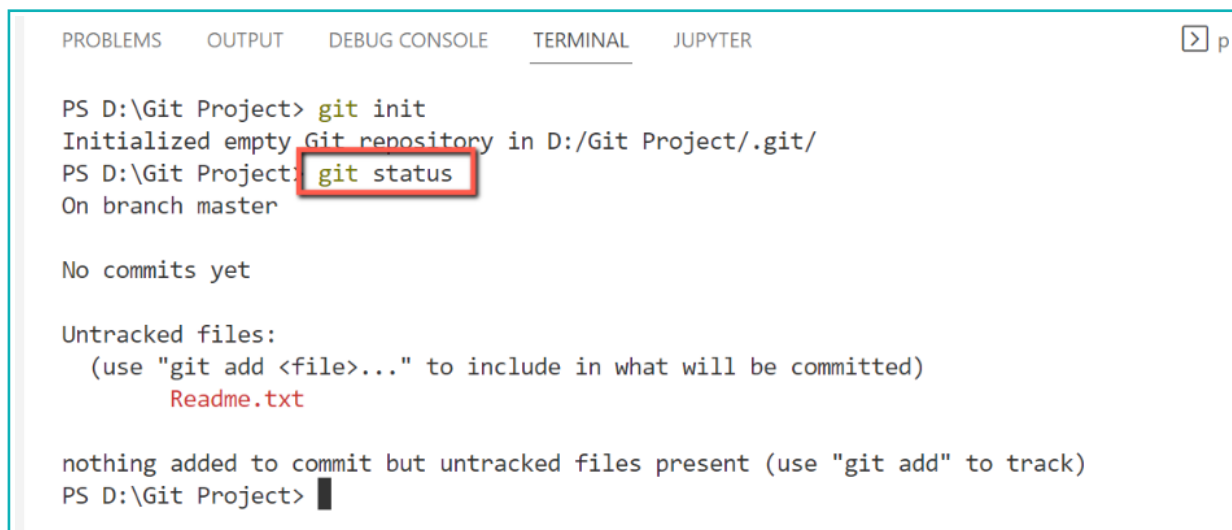
It is important to understand that if we make any changes to our 'Repo', 'Git' will not track those changes automatically. Only when we explicitly ask it to do so, will Git manage the changes made to those files. In that sense, files/folders present in a 'Git Repo' are either 'Tracked' or 'Untracked' by Git.

To summarise,

- Files that are present in the 'Git Repository' and are being monitored by 'Git' are considered '**Tracked Files**'.
- Files that are present in the 'Git Repository' but are not being monitored by 'Git' are considered '**Untracked Files**'.

To check the status of the files/folders present in the repository, 'Git' offers a simple command – **git status**.

- 1 Let us open the 'Terminal' and check the status of our repository using the command 'git status'.



```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  JUPYTER  [icon] p

PS D:\Git Project> git init
Initialized empty Git repository in D:/Git Project/.git/
PS D:\Git Project> git status
On branch master

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
       README.txt

nothing added to commit but untracked files present (use "git add" to track)
PS D:\Git Project> 
```

As can be seen, the 'git status' command reveals the following information about the 'Repo':

- Which branch we are in
- Whether any commits have been made
- Which files are untracked or tracked.

At this point, this information may not make much sense, but we will cover it all as we move on with the lesson.

File States in a Git Repository

Before we proceed further, we need to understand that a file within the 'Git Repo' might reside in one of the 3 states - 'modified', 'staged', and 'committed'.

Modified - Files that are modified but the changes have not yet been committed or recorded by 'Git'.

Staged - A file that has been modified and marked to be included in the next commit.

Committed - Changes made in the file have been saved/committed in the Repository.

So, by default, whenever we create a file/folder within a Repository, they are untracked. To make 'Git' track those files, the command 'git add' is used. This command specifies which files should be tracked by 'Git'.

Our working directory contains only a single file 'Readme.txt' and we will use the 'git add' command to allow 'Git' to track it.

- 2 Execute the command 'git add Readme.txt' on the Terminal.

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  JUPYTER
Untracked files:
PS D:\Git Project> git add Readme.txt
PS D:\Git Project>
PS D:\Git Project> |
```

- 3 To determine the state of our repository after executing the 'git add' command, we will again execute the command 'git status'.

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  JUPYTER
PS D:\Git Project> git add Readme.txt
PS D:\Git Project>
PS D:\Git Project> git status
On branch master

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
    new file:   Readme.txt
```

At this point, the 'Readme.txt' is in the 'Staged' state and this file will now be tracked, which means that any modifications made to this file will be monitored by 'Git'.

Exercise

 **Fill in the blanks.**

- 7** To initialize a local directory as a Git repository, we use the _____ command.
- 8** To check which files are tracked or what commits have been made in a Git repository, we can use the _____ command.

Committing Files

In simple terms, a **'commit'** can be referred to as a snapshot of all the changes made to the repository at a specific point in time.

It is important to understand that we cannot directly commit the changes in the repository. First, we need to stage the changes using the 'git add' command. This command will instruct 'Git' to track the files, after which, we can commit those changes. Only when we commit, all the changes made in the repository (like adding, removing, or modifying existing files) are recorded by 'Git'.

In simple terms, to record any change made in the repository to a commit, we first need to add it to the staging area (using the 'git add' command).

To issue a commit, we use the 'git commit' command. We can also specify a message while committing using the '-m field'. It is important to provide a clear and descriptive commit statement that explains what was changed and why.

Syntax – git commit -m 'your_message'

- 1** In the terminal, execute the command `git commit -m "Created Readme file with a single message"`.

Note:

We have already executed the 'git add' command in the previous step, so we can directly execute the 'git commit' command.



```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  JUPYTER

PS D:\Git Project> git commit -m 'Created Readme file with a single message'
Author identity unknown

*** Please tell me who you are.

Run

git config --global user.email "you@example.com"
git config --global user.name "Your Name"
```

Notice that we got an 'Author Identity Error'. This error occurs because to make a commit, Git requires the username and email address of whoever created the repository. To resolve the issue, simply run the commands shown in the error message and provide your email and name. So, let us resolve this error.

- 2 In the 'Terminal', execute the 'git config' commands and specify your email and name.



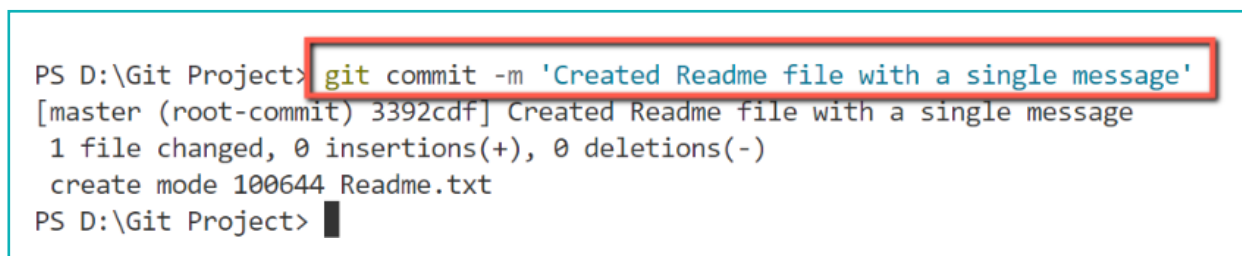
```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  JUPYTER

PS D:\Git Project> git config --global user.email "geek123@gmail.com"
PS D:\Git Project> git config --global user.name "Coder"
PS D:\Git Project>
PS D:\Git Project>
PS D:\Git Project>
```

Note:

In case, the error is not generated, you can skip the above step.

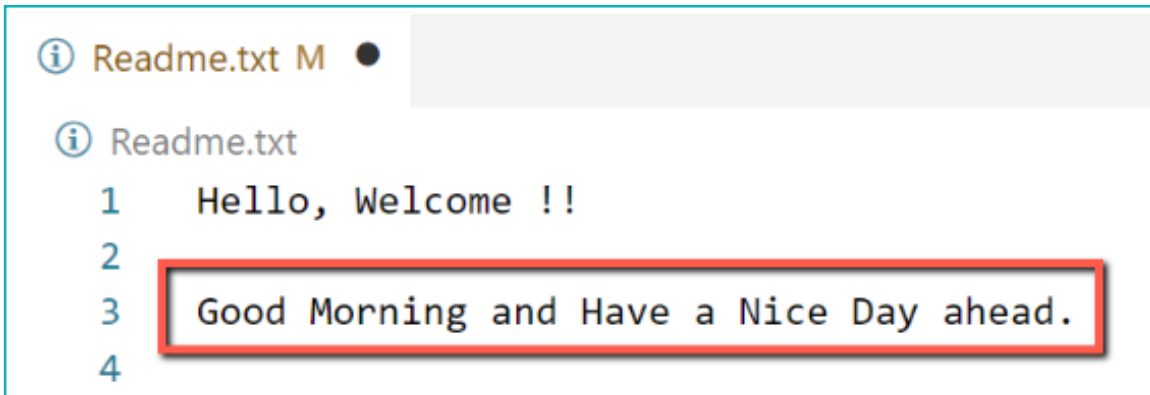
- 3 Execute the 'git commit' command now with the message.



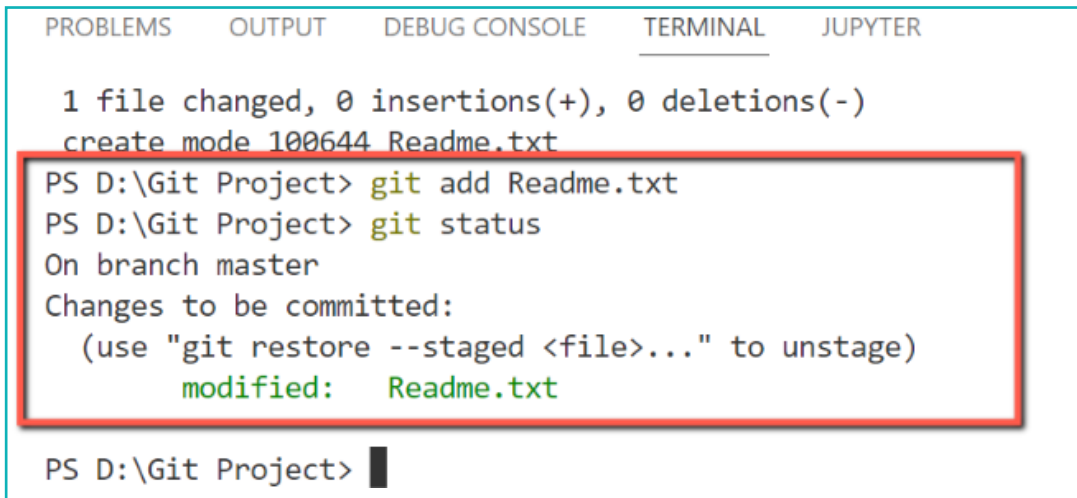
```
PS D:\Git Project> git commit -m 'Created Readme file with a single message'
[master (root-commit) 3392cdf] Created Readme file with a single message
1 file changed, 0 insertions(+), 0 deletions(-)
create mode 100644 Readme.txt
PS D:\Git Project>
```

To get a bit more familiar with the whole process, let us add some more content to the 'Readme.txt' file and commit the changes.

- 4 Add more content to the 'Readme' file.



- 5 Execute the 'git add' command so that the changes made in the file are available in the 'stage' area. After 'git add', you can also run the 'git status' command. This will display the message that the 'Readme' file is modified but the changes have not yet been committed.

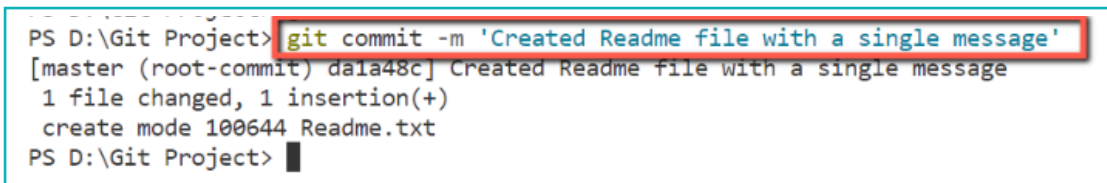


```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL JUPYTER

1 file changed, 0 insertions(+), 0 deletions(-)
create mode 100644 Readme.txt
PS D:\Git Project> git add Readme.txt
PS D:\Git Project> git status
On branch master
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        modified:   Readme.txt

PS D:\Git Project> |
```

- 6 Finally, commit the changes with a message.



```
PS D:\Git Project> git commit -m 'Created Readme file with a single message'
[master (root-commit) da1a48c] Created Readme file with a single message
1 file changed, 1 insertion(+)
create mode 100644 Readme.txt
PS D:\Git Project> |
```

In this way, we can keep a record of all the changes that we make within the repository.

View Commit History

- 1 'Git' allows us to view all the commit we made on a particular file using the '**git log**' command.

```
PS D:\Git Project> git log
commit 0560274dbb9db29d1a791f67b31415cff001949 (HEAD -> master)
Author: Coder <geek123@gmail.com>
Date: Mon Jun 19 11:31:02 2023 +0530

    Added another sentence on the file

commit 7f6743b71cb1ffb62fdcf953463ea23816fd7d8
Author: Coder <geek123@gmail.com>
Date: Mon Jun 19 11:29:04 2023 +0530

    Created Readme file with a single message
PS D:\Git Project>
```

Diagram illustrating the commit history output:

- The output shows two commits, ordered from newest to oldest.
- The top commit is labeled "Second Commit" with a red arrow pointing to it.
- The bottom commit is labeled "First Commit" with a red arrow pointing to it.

The output generated displays the 2 commits we had made, along with the time, author of the commit, and the commit message.

Exercise

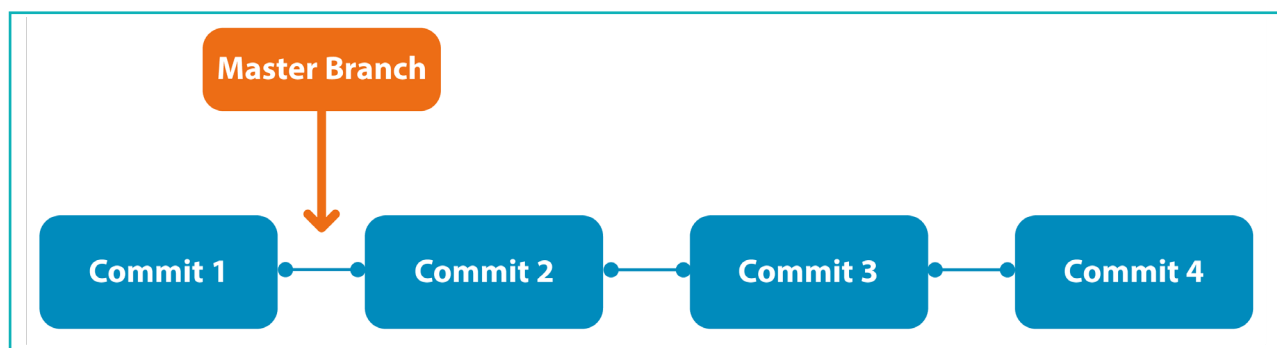
Tick mark ☒ the correct answer

- 9 Which command is used to set your email and username within a Git repository?
- | | | | |
|---------------|--------------------------|---------------|--------------------------|
| a. git init | ☐ | b. git config | ☐ |
| c. git status | ☐ | d. git add | ☐ |
- 10 Which command is used to view the history of all the commits made in the repository?
- | | | | |
|---------------|--------------------------|---------------|--------------------------|
| a. git log | ☐ | b. git config | ☐ |
| c. git status | ☐ | d. git add | ☐ |

Branching in Git

Consider a situation where you want to add a new functionality to your application but are worried that making changes to your existing code may result in an error, or you may want to develop the functionality separately, without affecting the main project. This is where **'Git Branches'** come in. 'Branching' in Git is a technique to continue working on a new feature or modification to an application without affecting the main part of the project.

As soon as we create a repository, 'Git' creates a default branch for our project. This branch is known as the **'Master'** or **'Main branch'**. Each project has only a single master branch that represents the main project.



Branches allow us to focus on distinct aspects of a project without affecting the main project. When the new functionality is finished developing, the new branch can be merged with the main branch.

For instance, if you want to add a new page to your website or add new functionality to your application, you may do so by creating a new branch dedicated to that page or functionality without affecting the main part of the project. Once you are done with creating the page or functionality, you can merge your changes from your branch into the primary branch.

Creating a New Branch

- 1 First, we will check the existing branches in our repository. So, execute the command **'git branch'** on the terminal. This command simply displays all the branches in your repository.

```
PS D:\Git Project>
PS D:\Git Project> git branch
* master
PS D:\Git Project> █
```

As we can see from the output, there is only one branch present – the 'master' branch.

- 2 To create a new branch, we can use the command `'git checkout -b <branch_name>'`.

```
PS D:\Git Project> git status
On branch master
nothing to commit, working tree clean
PS D:\Git Project> git checkout -b Index-page
Switched to a new branch 'Index-page'
PS D:\Git Project>
```

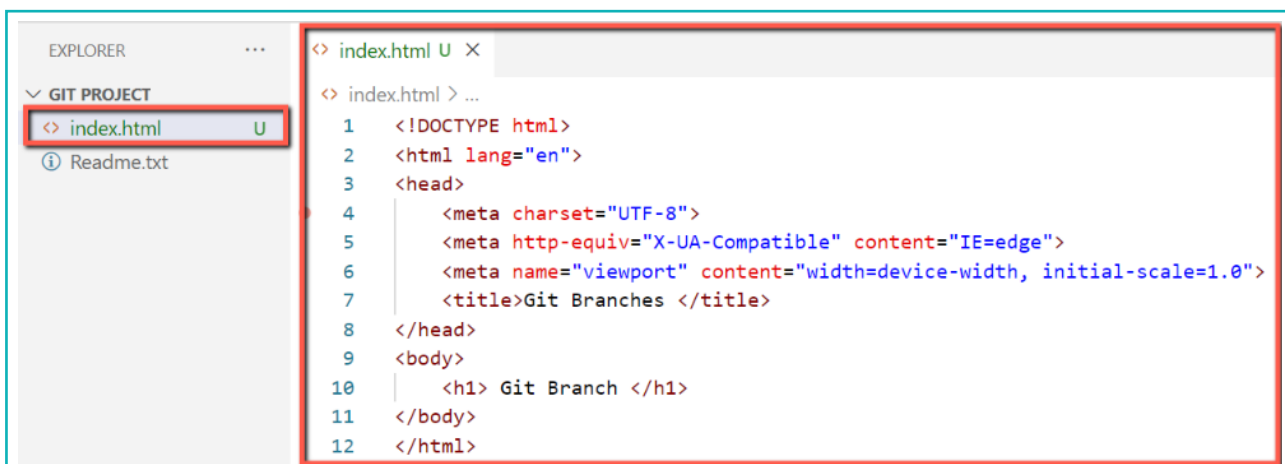
We are now in the 'Index-page' branch.

- 3 Now, if you execute the command `'git branch'`, it will show two branches. The '*' symbol before a branch name indicates the current working branch.

```
PS D:\Git Project> git branch
* Index-page
  master
PS D:\Git Project>
```

To see the practical implementation of branches, let us create an HTML page 'index.html' and manage it using the new branch 'Index-page'.

- 4 Create an HTML page 'index.html' and add the basic boilerplate along with a heading.



```
<> index.html U X
<> index.html > ...
1  <!DOCTYPE html>
2  <html lang="en">
3  <head>
4      <meta charset="UTF-8">
5      <meta http-equiv="X-UA-Compatible" content="IE=edge">
6      <meta name="viewport" content="width=device-width, initial-scale=1.0">
7      <title>Git Branch </title>
8  </head>
9  <body>
10     <h1> Git Branch </h1>
11 </body>
12 </html>
```

- 5 Add the updated changes to our project to the repository using the '**git add index.html**' command.

```
PS D:\Git Project> git add index.html
PS D:\Git Project> █
```

- 6 Commit the changes and add a relevant message.

```
PS D:\Git Project> git commit -m 'Created an HTML page'
[Index-page 1620526] Created an HTML page
1 file changed, 13 insertions(+)
create mode 100644 index.html
PS D:\Git Project> █
```

Switch over to the master branch using the command '**git checkout**'.

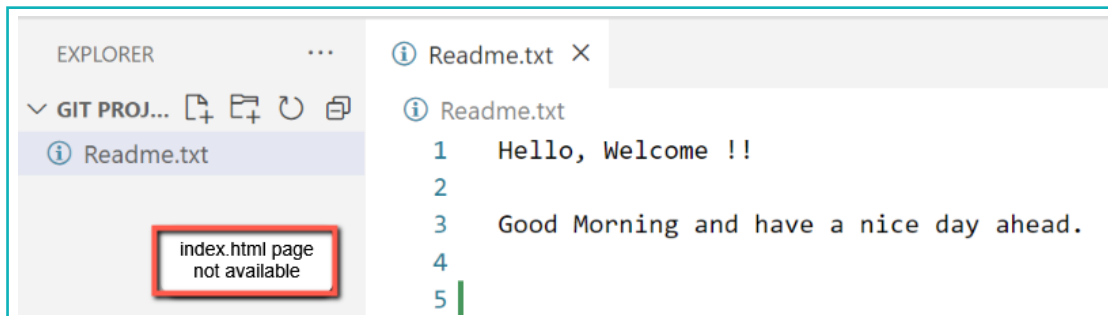
Switching Branches

As previously mentioned, all modifications made on a specific branch are only visible on that branch. So, let us test that by switching from the 'Index-page' branch to the master branch. To switch between branches, we can use the command '**git checkout <branch_name>**'.

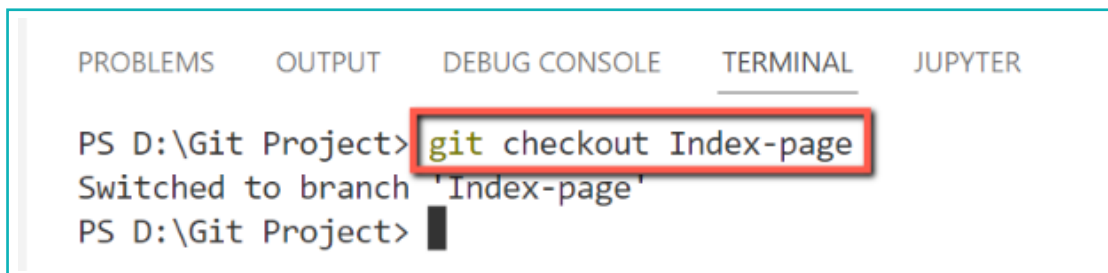
- 1 Switch over to the master branch using the command '**git checkout**'.

```
PS D:\Git Project> git checkout master
Switched to branch 'master'
PS D:\Git Project> █
```

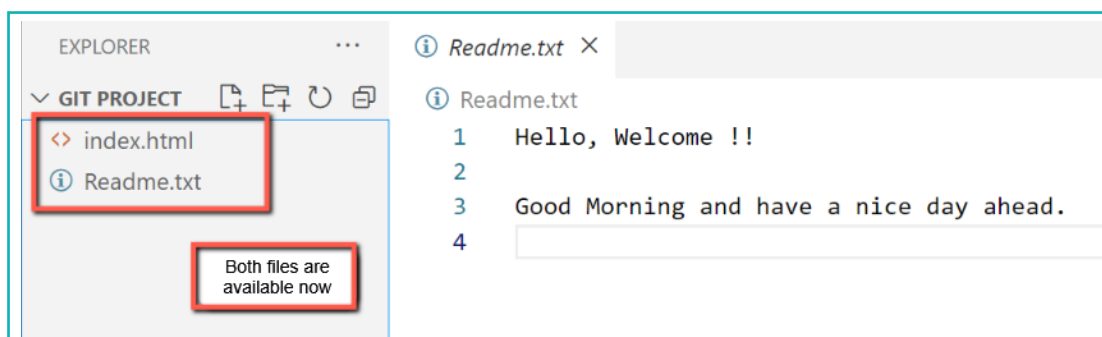
- 2 On executing the command, we shifted to the 'master branch'. Now, you will find that the 'index.html' page is no longer available. This is because that file was committed to a separate branch, the 'Index-page' branch, and so the 'master branch' has no idea about it.



- 3 Now, if you switch to the 'Index-page' branch, the 'index.html' will appear again.



- 4 Repository after switching to the 'Index-page' branch.



In this way, we can create multiple branches as and when required. Each branch will work independently of the other branch and will retain the changes applied to it.

Once we have completed all the modifications in the new branch, we can also merge the two branches using the git merge **<branch_name>** command.

- 5 First, switch to the main branch (the 'master branch') using the 'git checkout' command, and then execute the 'git merge' command.

```
PS D:\Git Project> git checkout master
PS D:\Git Project> git merge Index-page
Merge made by the 'ort' strategy.
index.html | 13 ++++++++
1 file changed, 13 insertions(+)
create mode 100644 index.html
```

The 'Index-page' branch is now merged with the 'master' branch. Now, all the changes we made in the 'Index-page' branch will also be available on the 'master' branch.

Exercise

Tick mark ☒ the correct answer

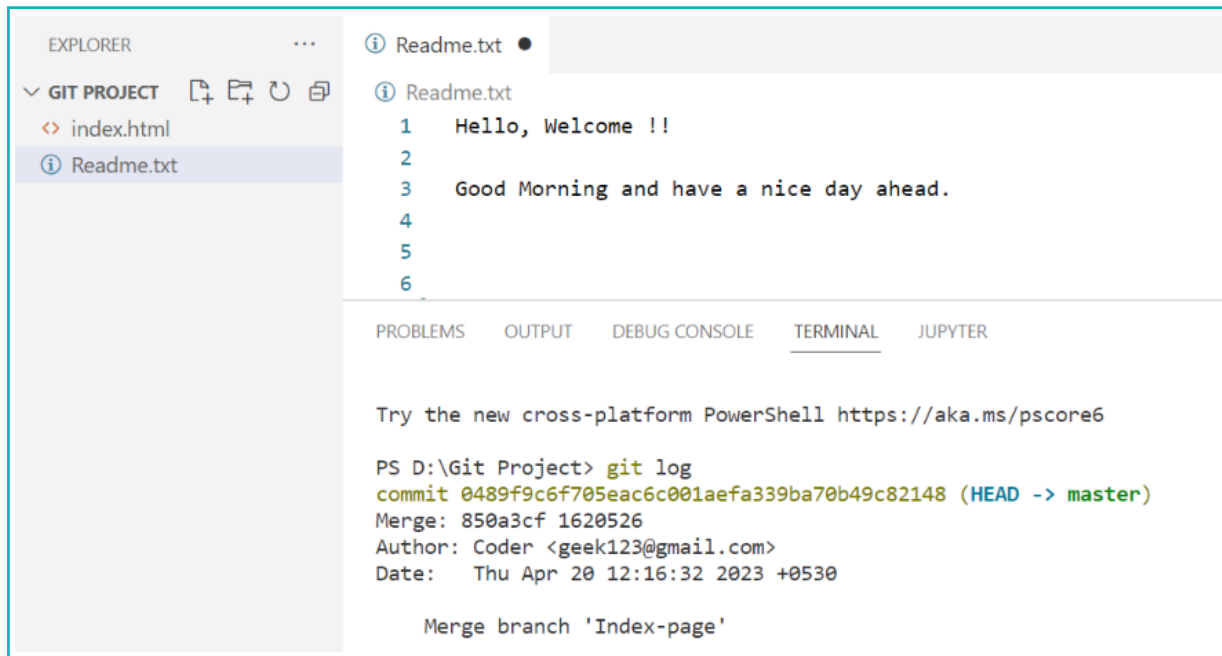
- 11 Which command is used to switch between branches?

a. git init	☐	b. git checkout	☐
c. git status	☐	d. git add	☐

- 12 Which command is used to view all the 'Git Branches'?

a. git branch	☐	b. git status	☐
c. git branching	☐	d. git merge	☐

Lesson Output



```
EXPLORER
  GIT PROJECT
    index.html
    Readme.txt

Readme.txt
1 Hello, Welcome !!
2
3 Good Morning and have a nice day ahead.
4
5
6

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL JUPYTER

Try the new cross-platform PowerShell https://aka.ms/pscore6

PS D:\Git Project> git log
commit 0489f9c6f705eac6c001aefa339ba70b49c82148 (HEAD -> master)
Merge: 850a3cf 1620526
Author: Coder <geek123@gmail.com>
Date: Thu Apr 20 12:16:32 2023 +0530

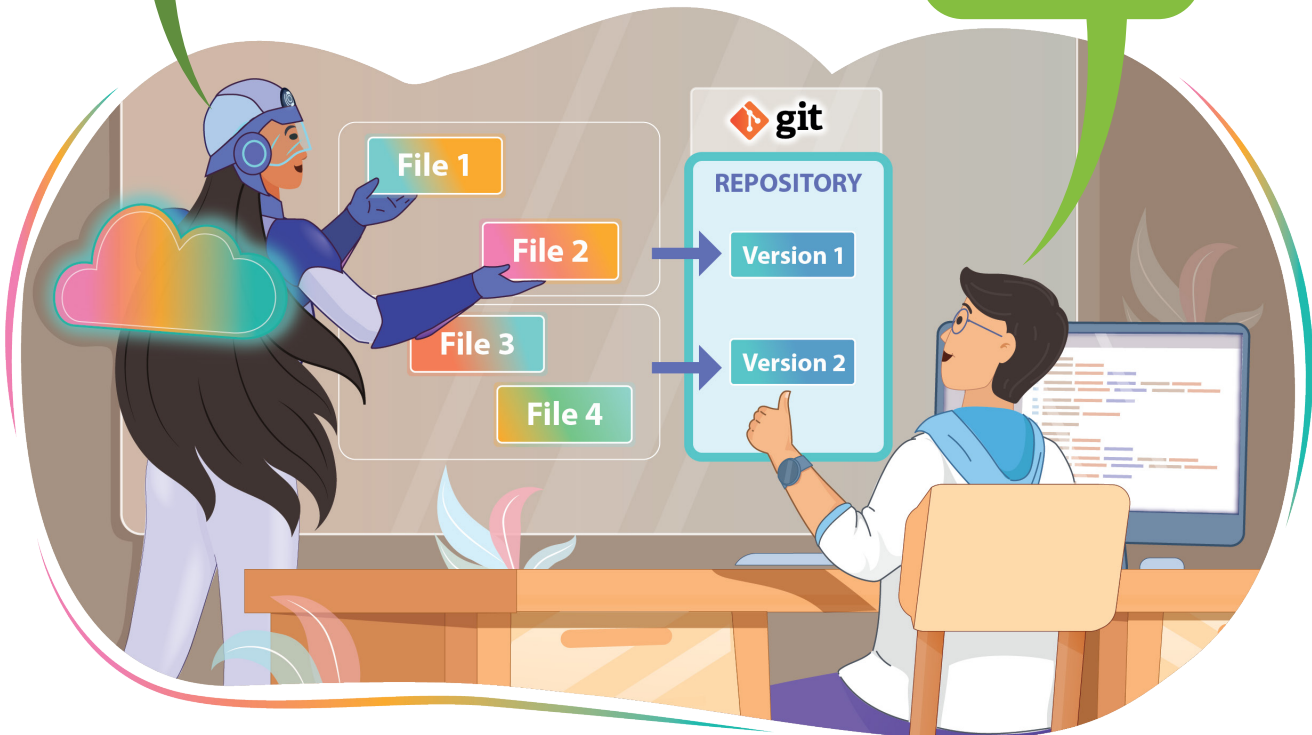
Merge branch 'Index-page'
```

In this lesson, we have learned about 'Git' and how we can create a local 'Git Repository' and keep track of all the changes made within it.

To summarise, 'Git' is a popular distributed version control system in software development that provides numerous benefits to developers. Some of the most significant benefits of 'Git' include, allowing multiple developers to work on the same codebase at the same time, maintaining a complete history of all changes made to the codebase, facilitating branching, and merging of code, providing an easy and reliable way to backup and recover code, and encouraging open-source collaboration. Overall, 'Git' is a must-have technology for modern software or app development since it offers a strong and adaptable approach to code management and collaboration with other developers.

Now that we know how to work with Git repositories and track file changes, we can collaborate with others more effectively. It's easier to manage different versions of files and merge changes seamlessly. Git really simplifies teamwork, don't you think?

Yes, Indeed..



Tech Flash

- **Version Control Systems** : Refers to a category of software tools that helps us in recording changes made to a file or a set of files so that we can keep track of all the changes or recall earlier functional versions, if required.
- **Open Source** : Applications whose source code are publicly accessible for anyone to view, modify and distribute, are referred to as 'Open-Source Applications'.
- **Git** : The most common 'Version Control System' for tracking changes in your codebase over time.